

Paragliding Intelligence Platform

Volume 15 - In-Flight Safety Assistant & Rescue Mode Architecture

Technical Specification v2 - Next-edge paragliding safety, live tracking, airspace, alerting and rescue operations

Document Field	Value
Status	Technical design draft for engineering, product and safety review
Primary readers	Frontend, mobile, backend, GIS, weather, ML, security and product teams
System scope	In-flight mode, safety assistant, tracking, emergency sharing, rescue dashboard and SAR support tools
Non-goal	The platform does not authorize flight, replace pilot judgement, replace official airspace sources or operate as an official emergency service
Dependencies	Volumes 01-14: PRD, SAD, DDD, API, Weather Engine, AI/ML, Mobile UX, Infrastructure, Security, QA, Business, Legal, Local Expert Ops and Hardware Stations

Table of Contents

- 1. Executive Summary
- 2. Product Scope and Safety Philosophy
- 3. User Roles and Operating Modes
- 4. In-Flight System Architecture
- 5. Mobile Sensor and Permission Architecture
- 6. In-Flight State Machine
- 7. Telemetry and Track Ingestion
- 8. On-Device Safety Engine
- 9. Cloud Safety Context Service
- 10. Alert Taxonomy and Human Factors
- 11. Airspace Guardian
- 12. Terrain, Glide and Landing Safety
- 13. Weather Nowcast and Hazard Monitoring
- 14. Emergency and Rescue Mode
- 15. Search-Area Estimation Model
- 16. APIs and WebSocket Contracts
- 17. Database Extensions
- 18. Offline-First Storage and Synchronization
- 19. Privacy, Consent and Legal Boundaries
- 20. Observability and Operations
- 21. QA and Acceptance Criteria
- 22. MVP/V1/V2 Roadmap
- 23. Engineering Backlog
- 24. References

1. Executive Summary

Volume 15 defines the in-flight safety and rescue layer for the paragliding meteo platform. The previous volumes explain how the platform predicts flyability, generates site forecasts, processes weather data, operates mobile UX, and protects user privacy. This volume turns those capabilities into a live companion used before launch, during flight and in emergency situations.

The core product thesis is simple: Windy-style maps help pilots understand weather, but a paragliding-first system should also monitor the live flight context and warn when the situation is becoming unsafe. The in-flight safety assistant uses location, altitude, heading, vertical speed, airspace, terrain, launch playbooks, live observations, radar/lightning, forecast updates, and pilot profile data to generate timely, explainable warnings.

- Primary value: reduce missed hazards such as airspace infringement, wind/gust increase, storm approach, loss of safe glide, cloudbase conflict, rotor exposure and poor landing options.
- Secondary value: allow trusted emergency contacts, club safety officers or rescue teams to see the last known context if the pilot requests help or fails a configured check-in.
- Commercial value: differentiates the product from generic weather apps by creating a safety operating system for paragliding, not just a map viewer.

Capability	Implementation summary	Safety boundary
Live tracking	Adaptive GNSS sampling, local buffering, batched upload, WebSocket fan-out	User controlled sharing and retention; no covert tracking
In-flight alerts	On-device rules plus cloud context; severity-based audio/visual/haptic notifications	Advisory only; pilot remains responsible
Airspace guardian	Offline airspace tiles, altitude checks, approach warnings and voice alerts	Airspace data must show source, age and confidence
Rescue mode	SOS session, emergency share link, last known position, drift/glide/search-area estimates	Not a substitute for calling local emergency number or official SAR coordination

2. Product Scope and Safety Philosophy

The safety assistant must be designed as decision support, not automatic flight control. Paragliding has high uncertainty: actual wind can diverge sharply from forecast, pilots have different skills, and mountain microclimates can change quickly. Therefore the system must avoid false certainty and must always show confidence, freshness and data-source provenance.

Principle	Technical requirement
Never hide uncertainty	Every recommendation must expose confidence, data age and missing data flags.
Prefer false caution over false green	If live data conflicts with forecast or data is stale, downgrade GO to MAYBE or NO-GO where appropriate.
On-device first for urgent warnings	Airspace, terrain and glide warnings must function during network loss using cached data.
No silent tracking	Live tracking and rescue sharing require explicit consent, visible status and fast disable controls.
No AI authority language	AI may summarize risk but must not claim conditions are safe, guaranteed or officially approved.

The core UX wording model is: GO, MAYBE, NO-GO for planning; in flight the wording must be action-oriented: Monitor, Caution, Land Soon, Land Now, Airspace Exit, Terrain Clearance, Network Lost, GPS Degraded, Rescue Session Active.

3. User Roles and Operating Modes

Role	Capabilities	Restrictions
Pilot	Start/stop tracking, receive safety alerts, create SOS session, share live link	Cannot access other pilots unless shared or club/event policy allows

Paragliding Intelligence Platform - Volume 15 - In-Flight Safety & Rescue Mode

Emergency contact	View shared live/rescue link, last known position, recent track, battery/network status	No editing; time-limited access token
Club safety officer	View opted-in club pilots, site conditions, local alerts and incident timeline	Club-scoped access only; audit logs required
Event safety officer	View competition pilots, task route, weather alerts, incident dashboard	Event time-boxed; disabled after event closure
Rescue operator	Access rescue session dashboard if invited/authorized by pilot/contact/club/event	Explicit rescue access with audit trail; legal review required per market

Mode	Trigger	Main UI	Backend services
Pre-launch safety check	Pilot opens launch detail or presses Start Flight	Checklist with wind, gust, rotor, airspace, cloudbase, weather trend	forecast-service, playbook-service, safety-context-service
In-flight mode	Pilot starts flight or auto-detected takeoff confirmed	Minimal high-contrast cockpit safety UI	track-ingestion, airspace-service, alert-orchestrator
Offline in-flight mode	Network loss or poor signal	Cached map, airspace, glide, landing and local alerts	Mobile SQLite, offline tile package
Emergency mode	SOS button, crash suspicion, overdue check-in or emergency contact escalation	Emergency card, share link, last known status	rescue-session-service, notification-service
Rescue operations	Authorized user opens rescue link/dashboard	Map, timeline, search-area estimate, export tools	rescue-dashboard, terrain-service, weather-context-service

4. In-Flight System Architecture

The in-flight architecture is split into mobile edge processing and cloud augmentation. Urgent warnings must be generated locally when possible; cloud services provide richer weather, live station data, radar/lightning, event monitoring and rescue dashboards.

Mobile App

- > Sensor Fusion Module
- > Offline Context Store: airspace, terrain, landing zones, site playbooks, forecast snapshot
- > On-Device Safety Engine
- > Alert UI / Voice / Haptics
- > Track Buffer
- > Sync Client
 - > API Gateway
 - > Track Ingestion Service
 - > Safety Context Service
 - > Alert Orchestrator
 - > Rescue Session Service
 - > WebSocket Fanout
 - > TimescaleDB + PostGIS + Redis + Object Storage

Service	Owns	Consumes	Publishes
track-ingestion-service	Track sessions, points, data-quality flags	Mobile track batches	track.point.received, track.session.updated
safety-context-service	Per-pilot in-flight context bundle	Forecast cubes, airspace, terrain, playbooks, live stations	safety.context.refreshed
on-device-safety-engine	Local alert decisions	Sensors, cached context, pilot config	safety.alert.local
alert-orchestrator	Cloud-generated warnings and notifications	Track events, radar/lightning, forecast deltas	safety.alert.cloud, push.notification.requested
rescue-session-service	Emergency sessions, rescue tokens, authorized views	SOS events, overdue check-ins, contacts	rescue.session.opened, rescue.session.updated
rescue-dashboard-service	Operator views and exports	Rescue session, terrain, weather, last known state	Audit events

5. Mobile Sensor and Permission Architecture

The mobile app must collect only the sensor data required for safety, tracking and user-enabled sharing. Background operation must follow platform rules. On iOS, background location updates require the location background mode and explicit programmatic enabling. On Android, continuous tracking must be implemented with the correct foreground service type and visible notification when active.

Sensor/input	Usage	Sampling rule	Offline requirement
GNSS location	Position, speed, heading, track line, airspace and glide checks	1 Hz during active flight if battery permits; degrade to 5-10 s on low battery	Required
Barometer	Altitude smoothing and vertical speed	1-5 Hz local only; upload derived altitude/vario, not raw high-frequency data	Strongly recommended
Accelerometer/gyroscope	Crash suspicion, movement state, takeoff/landing detection	Local high-frequency processing; upload event summaries only	Recommended
Compass/heading	Directional awareness and airspace approach context	Use OS fused heading when reliable	Optional
Battery/network	Rescue context and operational health	Upload with track batches and status events	Required
Microphone/camera	Not required for MVP safety assistant	Disabled by default; no background recording	Not used

Platform	Implementation note
iOS	Use Core Location with explicit user permission, background mode only during active tracking, visible in-app tracking status and a clear stop action.
Android	Use foreground service with location service type for active tracking, persistent notification, and Play policy-compatible background location justification.
Both	Provide a pre-flight permission diagnostic screen showing GPS, background permission, battery optimization, offline map package and emergency contact status.

6. In-Flight State Machine

A deterministic state machine reduces ambiguity and prevents noisy alerts. The mobile app may infer states, but the pilot must be able to override Start Flight and End Flight.

IDLE

- > PRELAUNCH_CHECK: user opens site / presses Start Flight
- > TRACKING_ARMED: permissions OK and offline package ready
- > AIRBORNE: speed/altitude/vertical movement indicates flight OR user confirms takeoff
- > SOARING: low ground speed + positive climb pattern
- > GLIDE: sustained sink/forward speed away from ridge/thermal
- > LANDING_APPROACH: low AGL near landing zone or decreasing altitude
- > LANDED_PENDING_CONFIRMATION: no movement + low AGL + track stable
- > CLOSED: user ends flight or auto-close after confirmation
- > EMERGENCY: SOS, crash suspicion, overdue check-in or external escalation

State	Entry signals	Allowed alerts
PRELAUNCH_CHECK	User selected site, forecast exists, local playbook loaded	No-go blockers, data stale, station offline, wind outside limits
AIRBORNE	Track started, speed/altitude movement confirms launch	Airspace, terrain, glide, storm, wind trend, cloudbase
SOARING	Climb rate positive and circular/low-speed behavior	Cloudbase, drift, airspace ceiling, overdevelopment
GLIDE	Sustained sink and forward movement	Glide-to-safe-landing, terrain clearance, valley wind, headwind
LANDING_APPROACH	Low AGL near landing or possible	Obstacle/terrain caution, wind at landing,

	outlanding	emergency contact check-in
EMERGENCY	SOS/crash suspicion/overdue	Minimize UI complexity, share status, conserve battery, guide call emergency services

7. Telemetry and Track Ingestion

Telemetry must be lightweight, resilient to network loss, and privacy-controlled. The app writes track points to local SQLite first, then uploads in compressed batches. The server stores high-resolution recent points in TimescaleDB and reduces older points according to retention policy.

```
{
  "track_id": "trk_01HX...",
  "seq": 1842,
  "sampled_at": "2026-06-05T11:42:18.432Z",
  "lat": 45.872134,
  "lon": 11.801244,
  "gps_alt_m": 1432.5,
  "baro_alt_m": 1427.1,
  "agl_m": 612.0,
  "speed_mps": 9.8,
  "heading_deg": 214,
  "vertical_speed_mps": 1.7,
  "accuracy_m": 6.2,
  "battery_pct": 63,
  "network": "lte",
  "quality_flags": ["BARO_OK", "GPS_OK"]
}
```

Mechanism	Rule
Upload batch size	10-60 points per request depending on network and battery state
Sequence numbers	Every point has monotonically increasing seq per track to support de-duplication
Idempotency	Client sends Idempotency-Key per batch; server ignores duplicates
Compression	Use gzip/zstd for track batches on mobile networks
Backpressure	If server rejects due to rate or quality, client stores local and retries with exponential backoff
Data minimization	Upload only required fields; raw IMU stays local except summary crash suspicion events

8. On-Device Safety Engine

The on-device safety engine runs even without network. It evaluates cached airspace, terrain, landing zones, pilot-configured limits, recent sensor history and last downloaded weather context. The engine should be deterministic, testable and versioned.

Rule group	Example trigger	Action
Airspace proximity	Pilot will intersect controlled/restricted airspace within 90 seconds at current track	Urgent voice: Airspace ahead, turn left/right if advisory direction is available
Ceiling limit	Altitude is within configurable margin of airspace ceiling or cloudbase limit	Caution or urgent depending on margin and climb rate
Glide safety	No known landing zone reachable with safety margin under current wind/headwind model	Land Soon / choose nearest safe field
Terrain clearance	Projected path intersects terrain buffer within 60-120 seconds	Critical terrain warning
Weather hazard	Cached radar/lightning/storm cell near	Land Soon / avoid direction of hazard

	route, or last cloud update indicates overdevelopment	
GPS degraded	Accuracy poor or stale for > configured threshold	Warn that airspace/terrain guidance is degraded
Battery critical	Battery below threshold during active tracking	Reduce sampling; notify pilot and emergency contact if configured

```
safety_decision = evaluate(
    track_window=last_120_seconds,
    pilot_profile=active_profile,
    offline_airspace=airspace_tile_index,
    offline_terrain=dem_tile_index,
    landing_zones=local_lz_index,
    weather_snapshot=last_synced_context,
    playbook_rules=site_rules,
    device_health={battery, gps_accuracy, network_state}
)
```

9. Cloud Safety Context Service

The cloud context service prepares compact context bundles for the mobile app and the cloud alert engine. A context bundle must be small enough to refresh over mobile networks and rich enough to support offline safety for the selected flying area.

Context bundle component	Source volume/service	Refresh cadence
Forecast snapshot	Volume 05 Weather Engine	Before flight and every 15-30 minutes when online
Live station observations	Volume 14 Sensor Network	1-5 minutes if stations are active
Airspace tiles	Volume 04 API / OpenAIP ingestion	Daily or on NOTAM/airspace update where available
Terrain tiles	Volume 05 terrain/DEM pipeline	Versioned static packages
Landing zones and hazards	Volume 13 local playbooks	When playbook version changes
Radar/lightning summary	Weather nowcast service	5-15 minutes depending on data source
Pilot limits	Profile/entitlement service	When profile changes

```
GET /v1/inflight/context?site_id=site_123&radius_km=80&profile_id=pilot_456
```

Response:

```
{
  "context_id": "ctx_20260605_1100_site_123",
  "expires_at": "2026-06-05T11:30:00Z",
  "airspace_package_url": "...",
  "terrain_package_url": "...",
  "weather_summary": {...},
  "playbook_version": "site_123:v17",
  "landing_zone_index_version": "lz_europe_2026_06_01",
  "safety_ruleset_version": "safety_rules_0.8.2"
}
```

10. Alert Taxonomy and Human Factors

Alerts must be rare, meaningful and actionable. Excessive alerts create alarm fatigue and can distract a pilot in turbulent air. The app must prioritize audio and haptic patterns during flight, with large visual cards only when the pilot can safely look at the phone.

Severity	Color/icon	Audio/haptic	Examples	Required user action
Info	Blue	Optional tone	GPS ready, tracking started, context updated	None

Paragliding Intelligence Platform - Volume 15 - In-Flight Safety & Rescue Mode

Caution	Yellow	Single short tone	Model disagreement high, wind increasing, GPS accuracy degraded	Monitor
Warning	Orange	Repeated tone until acknowledged	Airspace within 2 minutes, cloudbase margin low, landing options reduced	Change plan
Critical	Red	Distinct continuous pattern until safe/acknowledged	Terrain collision path, restricted airspace imminent, storm/lightning near route	Immediate action
Emergency	Red/white	SOS confirmation and contact notification	SOS active, crash suspicion, overdue check-in	Confirm status or seek help

- Every alert must contain: what happened, why it matters, confidence, freshness and recommended next action.
- Critical alerts must remain available offline and must not rely on AI-generated text.
- AI summaries may explain alert history after the flight, but active safety alerts must come from deterministic rules or reviewed models.
- Alerts must be suppressible only where safe; airspace and critical terrain alerts cannot be permanently muted during active flight.

11. Airspace Guardian

The airspace guardian checks horizontal and vertical proximity to controlled, restricted, danger and temporary airspaces. It must support offline airspace packages and display data provenance, age and confidence. openAIP can be used as one aeronautical data source, but national and official sources may be required in production per country.

Check	Implementation detail
Horizontal proximity	Buffer polygons by severity class and compute current/intercept distance with PostGIS/server and R-tree/on-device index.
Vertical proximity	Compare GPS/baro altitude against lower/upper airspace limits with altitude reference rules and conservative margins.
Projected conflict	Estimate 30/60/120 second projected path from current speed/heading and wind drift.
Ceiling warning	Warn before entering controlled airspace from below when climb rate suggests future conflict.
Offline package	Pre-download airspace polygons and metadata for selected flight radius and fallback country package.
Auditability	Store airspace alert events with source version, polygon ID, altitude and reason code.

```
airspace_conflict = intersects(
    projected_track_buffer,
    airspace_polygon
) and altitude_m between lower_limit_m - margin and upper_limit_m + margin
```

12. Terrain, Glide and Landing Safety

Terrain and glide checks answer the most important in-flight safety question: can the pilot safely reach a landing option or escape terrain under current conditions? This system is not a certified flight computer; it is an advisory layer designed for early warning and route awareness.

Subsystem	Inputs	Output
AGL estimator	GNSS/baro altitude, DEM tile, geoid/altitude reference model	Above-ground-level estimate and confidence
Glide ring	Wing class/pilot profile, current altitude,	Reachable area polygon

Technical Specification v2 | Safety decision support only - not official flight authorization or emergency dispatch

	estimated wind, safety margin	
Landing option ranker	Known landings, outlanding candidates, slope/landcover, wind direction	Ranked landing options
Terrain intercept	Projected path, speed, sink rate, terrain profile	Time-to-terrain caution/critical
Valley wind penalty	Forecast/live wind, terrain corridor, time of day	Reduced glide confidence or warning

```

reachable_distance_m = max(0, (agl_m - reserve_altitude_m)) * effective_glide_ratio
headwind_penalty = clamp(headwind_mps / pilot_speed_mps, 0, 0.45)
effective_glide_ratio = base_glide_ratio * (1 - headwind_penalty) * safety_factor

```

13. Weather Nowcast and Hazard Monitoring

The in-flight system must subscribe to weather deltas after launch. It should not rely only on the forecast snapshot downloaded at takeoff. The cloud alert engine monitors radar, lightning, live station observations, model updates and pilot reports near active tracks.

Hazard	Data inputs	Alert trigger
Wind/gust increase	Launch/landing stations, model update, pilot reports	Gust spread or sustained wind exceeds pilot/site limit
Storm/lightning	Lightning feed, radar, satellite/nowcast	Strike or storm cell within configured distance/time-to-route
Cloudbase conflict	Forecast/observation, climb rate, airspace ceiling	Pilot approaching cloudbase margin or cloudbase lowering
Overdevelopment	CAPE/instability, satellite/radar trend, local reports	Growth trend near route or launch sector
Rotor/foehn	Weather engine risk fields, station pressure/wind signatures, playbook rules	Risk threshold crosses warning level near terrain sector

```

event: weather.hazard.updated
{
  "hazard_type": "LIGHTNING_NEAR_ROUTE",
  "track_id": "trk_123",
  "distance_km": 9.4,
  "eta_minutes_to_route": 18,
  "severity": "CRITICAL",
  "source": "lightning_provider_x",
  "source_age_seconds": 55
}

```

14. Emergency and Rescue Mode

Rescue Mode is a controlled emergency-sharing workflow. It must make it easy to share the pilot context with trusted people while preventing location abuse. It is not an official emergency dispatch system unless integrated under formal agreements in a specific market.

Trigger	Workflow
Manual SOS	Pilot long-presses SOS, confirms if possible, app opens rescue session, shares link with emergency contacts and displays call emergency-services prompt.
Crash suspicion	Device detects abrupt stop/impact pattern and no movement; app asks pilot to cancel; if no response and user enabled this feature, contacts are notified.
Overdue check-in	Pilot configured check-in window expires; contact receives status request and last shared location if consented.
Event/club escalation	Competition or club safety officer flags pilot missing; rescue session created under event/club policy and audit trail.

Rescue dashboard component	Content
Last known state	Position, time, altitude, speed, heading, battery, network, GPS accuracy, last alert
Track timeline	Recent points, gaps, data quality, safety alerts, weather hazard events
Search-area estimate	Glide reachable area, wind drift estimate, terrain constraints, likely landing areas
Communication panel	Pilot contact status, emergency contact events, notes and export log
Export tools	KML/GeoJSON/PDF incident pack for authorized responders

```
POST /v1/rescue/sessions
{
  "track_id": "trk_123",
  "trigger": "MANUAL_SOS",
  "message": "Possible accident near ridge, need help",
  "share_with": ["emergency_contact_primary", "club_safety_officer"]
}
```

15. Search-Area Estimation Model

The search-area model is a decision-support model for authorized rescue views. It should estimate where the pilot might be after the last known point using terrain, altitude, glide performance, wind, drift, known landings and uncertainty. It must be clearly labelled as an estimate, not a definitive location.

Layer	Purpose
Last known point	Anchor the estimate and show confidence based on GPS accuracy and source age
Glide envelope	Estimate reachable area from altitude and conservative glide ratio
Wind drift envelope	Estimate likely drift direction/uncertainty after last known point
Terrain mask	Remove impossible terrain or highlight dangerous terrain rather than hiding it
Landing likelihood	Rank known landings, meadows/roads/flat areas if landcover data exists
Search grid	Generate priority zones A/B/C with rationale and uncertainty

```
for sample in monte_carlo_samples:
    wind = sample_wind(last_known_time, altitude_band, uncertainty)
    glide = sample_glide_ratio(pilot_profile, wing_class, uncertainty)
    sink = sample_sink_rate(weather_context, uncertainty)
    endpoint = propagate(last_known_point, heading, wind, glide, sink, terrain)
    accumulate_probability(endpoint)
```

The model can borrow concepts from SAR probability mapping and Monte Carlo uncertainty, but production use with authorities requires formal review, local legal assessment and training. The app should export data in simple formats such as GeoJSON/KML and never claim the estimated search area is official.

16. APIs and WebSocket Contracts

Endpoint/channel	Method	Owner service	Purpose
/v1/tracks	POST	track-ingestion-service	Start a track session
/v1/tracks/{track_id}/points:batch	POST	track-ingestion-service	Upload compressed telemetry batch
/v1/tracks/{track_id}	PATCH	track-ingestion-service	Update state, close track, add landing confirmation

Paragliding Intelligence Platform - Volume 15 - In-Flight Safety & Rescue Mode

/v1/inflight/context	GET	safety-context-service	Download offline/online context bundle
/v1/safety/events	POST	alert-orchestrator	Receive mobile-generated safety events
/v1/rescue/sessions	POST	rescue-session-service	Open emergency/rescue session
/v1/rescue/sessions/{id}/share-tokens	POST	rescue-session-service	Create time-limited emergency share link
/v1/rescue/sessions/{id}/search-area	GET	rescue-dashboard-service	Return estimated search area layers
/ws/v1/tracks/{track_id}	WebSocket	tracking-websocket-service	Live tracking stream for authorized viewers
/ws/v1/events/{event_id}/tracking	WebSocket	event-service	Competition/event tracking dashboard

POST /v1/tracks/{track_id}/points:batch

Headers:

Authorization: Bearer <token>

Idempotency-Key: trk_123_batch_184

Body:

```
{
  "client_batch_id": "batch_184",
  "points": [ ... ],
  "compression": "none",
  "device_state": {"battery_pct": 63, "network": "lte"}
}
```

Response:

```
{
  "accepted": 42,
  "duplicate": 0,
  "rejected": 0,
  "next_upload_hint_seconds": 5
}
```

WebSocket message: track.point

```
{
  "type": "track.point",
  "track_id": "trk_123",
  "lat": 45.872134,
  "lon": 11.801244,
  "alt_m": 1432,
  "speed_mps": 9.8,
  "heading_deg": 214,
  "sampled_at": "2026-06-05T11:42:18Z",
  "quality": "GOOD"
}
```

17. Database Extensions

These schemas extend Volume 03. Track points should be TimescaleDB hypertables; geometry should use PostGIS. Emergency access tokens must be short-lived and auditable.

```
CREATE TABLE flight_tracks (
  id UUID PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES users(id),
  site_id UUID REFERENCES launch_sites(id),
  status TEXT NOT NULL CHECK (status IN ('ARMED','AIRBORNE','LANDED','CLOSED','EMERGENCY')),
  started_at TIMESTAMPTZ NOT NULL,
  ended_at TIMESTAMPTZ,
  visibility TEXT NOT NULL DEFAULT 'private',
```

```

device_id UUID,
created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE track_points (
  track_id UUID NOT NULL REFERENCES flight_tracks(id),
  sampled_at TIMESTAMPTZ NOT NULL,
  seq BIGINT NOT NULL,
  geom GEOGRAPHY(Point,4326) NOT NULL,
  gps_alt_m DOUBLE PRECISION,
  baro_alt_m DOUBLE PRECISION,
  agl_m DOUBLE PRECISION,
  speed_mps DOUBLE PRECISION,
  heading_deg DOUBLE PRECISION,
  vertical_speed_mps DOUBLE PRECISION,
  accuracy_m DOUBLE PRECISION,
  battery_pct INTEGER,
  quality_flags TEXT[],
  PRIMARY KEY(track_id, sampled_at, seq)
);
-- SELECT create_hypertable('track_points', 'sampled_at');
CREATE INDEX track_points_geom_gix ON track_points USING GIST (geom);
CREATE INDEX track_points_track_time_idx ON track_points (track_id, sampled_at DESC);

CREATE TABLE safety_events (
  id UUID PRIMARY KEY,
  track_id UUID REFERENCES flight_tracks(id),
  user_id UUID REFERENCES users(id),
  event_type TEXT NOT NULL,
  severity TEXT NOT NULL CHECK (severity IN ('INFO','CAUTION','WARNING','CRITICAL','EMERGENCY')),
  reason_code TEXT NOT NULL,
  geom GEOGRAPHY(Point,4326),
  occurred_at TIMESTAMPTZ NOT NULL,
  source TEXT NOT NULL,
  payload JSONB NOT NULL DEFAULT '{}::jsonb'
);

CREATE TABLE rescue_sessions (
  id UUID PRIMARY KEY,
  track_id UUID NOT NULL REFERENCES flight_tracks(id),
  opened_by_user_id UUID REFERENCES users(id),
  trigger TEXT NOT NULL,
  status TEXT NOT NULL CHECK (status IN ('OPEN','ACKNOWLEDGED','CLOSED','FALSE_ALARM')),
  opened_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  closed_at TIMESTAMPTZ,
  summary JSONB NOT NULL DEFAULT '{}::jsonb'
);

CREATE TABLE rescue_share_tokens (
  id UUID PRIMARY KEY,
  rescue_session_id UUID NOT NULL REFERENCES rescue_sessions(id),
  token_hash TEXT NOT NULL,
  scope TEXT NOT NULL,
  expires_at TIMESTAMPTZ NOT NULL,
  revoked_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

18. Offline-First Storage and Synchronization

Local store	Contents	Retention
-------------	----------	-----------

Paragliding Intelligence Platform - Volume 15 - In-Flight Safety & Rescue Mode

SQLite track buffer	Unsynced track points, event queue, upload cursor	Until upload confirmed and user retention policy allows deletion
Offline map package	Vector tiles, terrain tiles, airspace tiles, landing zones	Versioned; refreshed before flight
Weather context cache	Forecast snapshot, live station last values, radar/lightning summary	Expires rapidly; stale flag shown
Safety ruleset cache	Deterministic rules and thresholds	Versioned; must support rollback
Emergency contact cache	Minimal contact routing IDs and display names	User-controlled

- Sync must be append-only for track points to avoid destructive conflicts.
- If the network returns after an SOS event, emergency session updates must be prioritized over normal telemetry.
- If the server ruleset is newer than the mobile ruleset, the app downloads the update but must not interrupt active flight unless the update fixes a critical issue.
- Offline packages must show version, source age and coverage radius before launch.

19. Privacy, Consent and Legal Boundaries

In-flight safety uses sensitive location data. The product must be built around explicit consent and narrow access. This chapter complements Volume 09 Security and Volume 12 Legal/Safety/Data Licensing.

Control	Requirement
Tracking consent	Pilot must explicitly start tracking or opt into auto-start; status must be visible while active
Emergency sharing	Share links are time-limited, scoped, revocable and logged
Club/event visibility	Pilot must know when club/event operators can see live position
Location retention	Default private tracks may be downsampled or deleted after configured retention; emergency records follow legal policy
AI usage	Personal location and emergency data must not be used for model training unless separately consented and anonymized
Rescue disclaimer	Every rescue view must state: call local emergency services first; app data is supplemental

20. Observability and Operations

Metric	Target / alert
Track point ingest p95 latency	< 2 seconds for online users in primary region
WebSocket fanout latency p95	< 3 seconds for authorized viewers
Push emergency notification creation	< 10 seconds after SOS accepted by backend
Safety context bundle generation	< 5 seconds p95 for 80 km radius MVP package
False emergency escalation rate	Tracked and reviewed; high false-positive features must remain opt-in
Critical alert delivery failure	Page on-call if backend-generated critical alerts fail above threshold

- Operational dashboards must separate normal telemetry load from emergency-session load.
- Emergency sessions must be easy to locate for support staff, but support access requires strict RBAC and audit logs.
- Incident review must record data gaps: GPS loss, network loss, stale airspace, stale forecast, station outage, model disagreement.
- Backups and disaster recovery procedures from Volume 08 must include rescue session records and audit logs.

21. QA and Acceptance Criteria

Test area	Acceptance criteria
Offline tracking	Track points are buffered for at least 4 hours offline and uploaded without duplicates after reconnect
Airspace alert	Known test route toward restricted airspace triggers warning

Paragliding Intelligence Platform - Volume 15 - In-Flight Safety & Rescue Mode

	before threshold and logs source polygon ID
Terrain alert	Synthetic route toward terrain collision triggers critical alert within configured time-to-terrain window
SOS workflow	Manual SOS creates rescue session, sends configured notifications and opens share dashboard in under target latency
Crash suspicion	Feature remains opt-in; test harness validates cancel workflow and false-positive handling
Privacy	Revoked share token immediately blocks dashboard and is written to audit log
Battery	Low-battery mode reduces sampling and warns pilot without stopping active safety checks
AI guardrails	AI briefing cannot generate emergency instructions beyond approved templates and cannot override deterministic alerts

Example automated test:

Given offline airspace package version v2026_06

And pilot trajectory projected into polygon ASP_789 within 75 seconds

When on-device safety engine evaluates the last 30 seconds of points

Then it emits severity=WARNING reason_code=AIRSPACE_PROJECTED_CONFLICT

And the alert payload contains polygon_id=ASP_789, source_version=v2026_06 and confidence >= 0.8

22. MVP/V1/V2 Roadmap

Phase	Included
MVP	Manual start/stop tracking, offline track buffer, basic airspace warnings, emergency share link, last known position, launch safety checklist
V1	Live tracking links, cloud weather hazard alerts, glide-to-known-landing, audio warnings, event/club dashboard
V2	Advanced terrain escape, Monte Carlo search-area estimate, crash suspicion opt-in, watch integration, station-driven wind alerts
V3	Authority integrations where legally approved, richer rescue exports, in-flight voice assistant, machine-learned pilot risk personalization

23. Engineering Backlog

Epic	Stories
Mobile tracking core	Build state machine, local SQLite track buffer, adaptive sampling and upload client
Offline context packages	Package airspace, terrain, landings, playbook rules and weather snapshot for selected site/radius
Safety engine	Implement deterministic rule runner, severity taxonomy, alert UI and voice/haptic delivery
Backend ingestion	Build track batch endpoint, idempotency, TimescaleDB persistence and WebSocket fanout
Rescue sessions	Build SOS API, share tokens, emergency contact notification and rescue dashboard
Airspace guardian	Implement offline spatial index, projection checks, altitude limits and audit logging
QA harness	Synthetic flight routes, offline network simulator, alert golden tests and privacy tests

24. References

The following primary references informed implementation constraints and should be reviewed again before production release, because mobile OS policies, app-store requirements and aviation data terms can change.

Technical Specification v2 | Safety decision support only - not official flight authorization or emergency dispatch

Paragliding Intelligence Platform - Volume 15 - In-Flight Safety & Rescue Mode

- Apple Developer Documentation - Core Location background updates:
<https://developer.apple.com/documentation/corelocation/handling-location-updates-in-the-background>
- Apple Developer Documentation - allowsBackgroundLocationUpdates:
<https://developer.apple.com/documentation/corelocation/cllocationmanager/allowsbackgroundlocationupdates>
- Android Developers - Foreground service types:
<https://developer.android.com/develop/background-work/services/fgs/service-types>
- Android Developers - Location permissions: <https://developer.android.com/develop/sensors-and-location/location/permissions>
- Google Play Help - Background location permissions:
<https://support.google.com/googleplay/android-developer/answer/9799150>
- openAIP API Documentation: <https://docs.openaip.net/>
- openAIP aeronautical data platform: <https://www.openaip.net/>
- IMO - IAMSAR Manual overview: <https://www.imo.org/en/ourwork/safety/pages/iamsarmanual.aspx>